



TC2038 - Análisis y diseño de algoritmos avanzados

Algoritmos de Enrutamiento Óptimo: Dijkstra y Floyd-Warshall

Profesor - Alison Muñoz Capote
Computación - Facultad de Ingeniería

Mapa del Tema y Objetivos

Objetivo de la Sesión:

Comprender y dominar los algoritmos de caminos mínimos (Shortest Path) en grafos: **Dijkstra** y **Floyd-Warshall**. Analizaremos sus bases teóricas, diferencias estructurales, ejecución paso a paso y escenarios de aplicación en la industria tecnológica.

Índice:

1. Algoritmo de Dijkstra: Motivación y Teoría
2. Dijkstra: Ejercicios de Aplicación
3. Algoritmo de Floyd-Warshall: Motivación y Teoría
4. Floyd-Warshall: Ejercicios de Aplicación
5. Tabla Comparativa y Conclusiones
6. Asignaciones (Preparación para NP-Hard)

1. Algoritmo de Dijkstra

El Rescate a Contrarreloj

Situación Problema

Eres el desarrollador principal del sistema de despacho del 911. Una ambulancia está en la base (Nodo A) y debe llegar al lugar de un accidente (Nodo Z) lo más rápido posible.

La ciudad es una red de calles (Aristas) e intersecciones (Nodos). Cada calle tiene un tiempo estimado de tránsito en minutos (Pesos). El tráfico cambia constantemente, por lo que no puedes confiar en una ruta precalculada ayer.

¿Por qué no usar Búsqueda en Anchura (BFS)?

Una Búsqueda en Anchura (BFS) estándar explora por *saltos* (número de calles). Si usamos BFS, el sistema podría enviar la ambulancia por una ruta de 2 calles principales que están completamente congestionadas (30 min), ignorando una ruta de 4 calles secundarias vacías (10 min).

Necesitamos un algoritmo consciente del *costo* o *peso* de cada arista.

La Base Teórica de Dijkstra

El Enfoque *Greedy*

El algoritmo de Edsger W. Dijkstra (1956) encuentra el camino más corto desde **un origen único** hacia todos los demás nodos del grafo.

Su filosofía es voraz: en cada paso, confía ciegamente en la información que tiene y **siempre elige explorar el nodo más cercano** que aún no ha sido visitado.

Estructura fundamental:

Utiliza una **Cola de Prioridad** para mantener ordenados los nodos según su costo acumulado desde el origen.

El Proceso de Relajación

El corazón matemático de Dijkstra es la **Relajación de Aristas**.

Imagina que crees que la mejor forma de llegar a la intersección V toma 20 minutos. De pronto, explorando el nodo U (al que llegas en 10 min), ves que hay una calle directa de U a V que toma 5 min.

Costo actual a V : 20

Nuevo costo propuesto ($U + \text{calle}$): $10 + 5 = 15$

Como $15 < 20$, **relajamos** (actualizamos) el valor de V a 15. Sobrescribimos la historia con una ruta mejor.

Restricción Crítica de Dijkstra

PROHIBIDOS LOS PESOS NEGATIVOS

Dijkstra asume que **viajar siempre cuesta algo** (tiempo, distancia, gasolina). Si una calle tuviera un costo negativo (ej. *te pagan por pasar por aquí*), la suposición voraz de que *un nodo visitado ya tiene su ruta óptima definitiva* se rompe por completo.

Si hay pesos negativos, Dijkstra dará respuestas incorrectas.

Dijkstra: Ejercicio 1 (La Ambulancia)

El Grafo de la Ciudad:

- Nodos: A (Base), B, C, D (Accidente)
- Calles (Bidireccionales) y tiempos:
 - A-B: 2 min
 - A-C: 5 min
 - B-C: 1 min
 - B-D: 7 min
 - C-D: 3 min

Objetivo: Camino mínimo de A hacia D.

Ejercicio 1: Inicialización

Tabla de Estado Inicial. El origen es **A**. Todo lo demás es infinito (∞).

Nodo	Costo Total	Nodo Previo	¿Visitado?
A (Base)	0	-	No
B	∞	-	No
C	∞	-	No
D (Destino)	∞	-	No

Ejercicio 1: Iteración 1

Nodo actual: A (Costo: 0)

Vecinos no visitados: B (peso 2), C (peso 5)

- Ruta A $\rightarrow$ B: $0 + 2 = \mathbf{2}$. ($2 < \infty$) $\rightarrow$ Actualizamos B.
- Ruta A $\rightarrow$ C: $0 + 5 = \mathbf{5}$. ($5 < \infty$) $\rightarrow$ Actualizamos C.

Marcamos **A** como Visitado.

Nodo	Costo Total	Nodo Previo	¿Visitado?
B	2	A	No
C	5	A	No

Ejercicio 1: Iteración 2

Nodo actual: B (Es el menor no visitado, Costo: 2)

Vecinos no visitados: C (peso 1), D (peso 7)

- Ruta $A \rightarrow B \rightarrow C$: $2 + 1 = \mathbf{3}$. ($3 < 5$) -> **iRelajación!** Actualizamos C.
- Ruta $A \rightarrow B \rightarrow D$: $2 + 7 = \mathbf{9}$. ($9 < \infty$) -> Actualizamos D.

Marcamos **B** como Visitado.

Nodo	Costo Total	Nodo Previo	¿Visitado?
C	3	B	No
D	9	B	No

Ejercicio 1: Iteraciones 3 y Final

Nodo actual: C (Menor no visitado, Costo: 3)

Vecinos no visitados: D (peso 3)

- Ruta $\rightarrow C \rightarrow D$: $3 + 3 = \mathbf{6}$. ($6 < 9$) -> **iRelajación!** Actualizamos D.
- Marcamos **C** como Visitado.

Nodo actual: D (Destino alcanzado y sin vecinos a explorar). Marcamos D como visitado. Fin.

Ejercicio 1: Resultado

Para reconstruir la ruta de la ambulancia, leemos la tabla desde el Destino hacia atrás buscando los Nodos Previos.

Nodo	Costo Total	Nodo Previo
D	6	C
C	3	B
B	2	A

Ruta final: A → B → C → D.

Tiempo total: 6 minutos.

Dijkstra: Ejercicio 2 (Redes IP)

Enrutamiento de Paquetes en Servidores (Grafo Dirigido)

Calcula el camino más corto desde el **Router 1** hacia el **Router 4**. (Nota: los enlaces ahora tienen una sola dirección).

- $1 \rightarrow 2$ (Costo 4)
- $1 \rightarrow 3$ (Costo 1)
- $3 \rightarrow 2$ (Costo 2)
- $2 \rightarrow 4$ (Costo 5)
- $3 \rightarrow 4$ (Costo 8)

Ejercicio 2: Resolución Rápida

1. **Inicio:** 1 (Costo 0).
2. **Desde 1:** Relajamos 2 (costo 4, previo 1) y 3 (costo 1, previo 1). *Visitamos 1.*
3. **Siguiente menor (3):** Vecinos 2 y 4.
 - A 2: Costo de $3(1) + \text{Enlace}(2) = 3$. ($3 < 4$). Relajamos 2. Previo es 3.
 - A 4: Costo de $3(1) + \text{Enlace}(8) = 9$. Relajamos 4. Previo es 3.
 - *Visitamos 3.*

Ejercicio 2: Finalización

4. **Siguiente menor (2):** (Su costo ahora es 3). Vecinos: 4.

- A 4: Costo de $2(3) + \text{Enlace}(5) = 8$. ($8 < 9$). Relajamos 4. Previo es 2.
- *Visitamos 2.*

5. **Nodo 4:** Visitado. Fin.

Ruta óptima reconstruida: $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$

Latencia Total: 8 ms.

2. Algoritmo de Floyd-Warshall

La Matriz Logística Universal

Situación Problema

Eres el Arquitecto de Software de FedEx. Tienes 50 centros de distribución en todo el país. El sistema de cotización web necesita saber la distancia mínima entre **cualquier par de centros al instante**, para mostrarle el precio de envío al cliente antes de que cierre la página.

Correr Dijkstra 50 veces (una por cada centro) cada vez que el mapa se actualiza es ineficiente. Necesitas pre-calcular toda la matriz de una vez.

La Base Teórica de Floyd-Warshall

Programación Dinámica

El algoritmo de Robert Floyd (1962) calcula la ruta más corta entre **todos los pares de nodos simultáneamente**.

En lugar de ser *voraz*, utiliza Programación Dinámica: descompone el problema enorme en subproblemas incrementales, utilizando una matriz bidimensional (tabla) que se va refinando paso a paso.

El Nodo Intermediario **K**

Floyd-Warshall se basa en una simple pero poderosa pregunta iterativa:

"Si quiero ir del nodo i al nodo j , ¿me sale más barato dar un rodeo pasando obligatoriamente por un nodo intermedio K ?"

Matemáticamente se expresa así:

$$D[i][j] = \min(D[i][j], \quad D[i][k] + D[k][j])$$

Soportando lo Imposible: Pesos Negativos

A diferencia de Dijkstra, **Floyd SÍ soporta aristas con pesos negativos.**

Como evalúa todas las permutaciones posibles de nodos intermedios de manera estructurada, no se deja engañar si una ruta de repente *devuelve* costo.

(Nota: Falla si existe un ciclo negativo cerrado, es decir, un bucle infinito que siga restando costo infinitamente).

Floyd-Warshall: Ejercicio 1 (La Red)

Grafo de 3 Nodos (Dirigido):

1, 2 y 3.

- $1 \rightarrow 2$ (Peso 8)
- $2 \rightarrow 1$ (Peso 2)
- $1 \rightarrow 3$ (Peso 5)
- $3 \rightarrow 2$ (Peso 1)

Vamos a calcular todas las distancias entre todos los pares.

Ejercicio 1: Matriz Inicial (K=0)

Construimos la matriz inicial D^0 solo con las aristas directas. La diagonal siempre es 0. Lo que no tiene conexión directa es ∞ .

Origen \ Destino	1	2	3
1	0	8	5
2	2	0	∞
3	∞	1	0

Ejercicio 1: Usando $K = 1$

Fijamos el Nodo 1 como intermediario. ¿Puede el nodo 1 ser un atajo para alguien?

Evaluamos $D[2][3]$ (De 2 a 3 pasando por 1):

$$D[2][1] + D[1][3] = 2 + 5 = 7.$$

Como $7 < \infty$, actualizamos.

El resto de celdas no mejoran al pasar por 1.

Origen \ Destino	1	2	3
2	2	0	7

(Las demás filas se mantienen igual)

Ejercicio 1: Usando $K = 2$ y $K = 3$

$K = 2$ (Nodo 2 como intermediario):

¿Mejora $3 \rightarrow 1$ pasando por 2?

$D[3][2] + D[2][1] = 1 + 2 = 3$. ($3 < \infty$). Actualizamos.

$K = 3$ (Nodo 3 como intermediario):

¿Mejora $1 \rightarrow 2$ pasando por 3?

$D[1][3] + D[3][2] = 5 + 1 = 6$. ($6 < 8$). ¡Relajamos!

Ejercicio 1: Resultado Final

Después de iterar K para todos los nodos (1, 2 y 3), obtenemos la matriz definitiva. Esta tabla contiene la ruta más corta para cualquier consulta.

Origen \ Destino	1	2	3
1	0	6	5
2	2	0	7
3	3	1	0

Floyd-Warshall: Ejercicio 2 (Pesos Negativos)

Arbitraje Financiero

Imagina un grafo de intercambio de divisas donde las aristas representan el costo de transacción, pero algunas ofrecen bonos (pesos negativos).

- $A \rightarrow B$ (Costo 3)
- $B \rightarrow C$ (Costo -2)
- $A \rightarrow C$ (Costo 5)

Calculemos con Floyd.

Ejercicio 2: Resolución

Matriz Inicial ($K=0$):

$A \rightarrow B = 3$, $B \rightarrow C = -2$, $A \rightarrow C = 5$.

Iteración clave (Usando $K=B$ como intermediario):

Evaluamos la ruta $A \rightarrow C$ pasando por B :

Costo $A \rightarrow B$ (3) + Costo $B \rightarrow C$ (-2) = **1**.

Comparamos: $1 < 5$.

Actualizamos $A \rightarrow C$ a **1**.

El algoritmo manejó perfectamente el peso negativo, descubriendo que triangular por B era 4 unidades más barato que el enlace directo.

3. Comparación y Conclusiones

Dijkstra vs Floyd-Warshall

Característica	Dijkstra	Floyd-Warshall
Propósito	Un Origen a Todos	Todos a Todos
Paradigma	Greedy (Voraz)	Programación Dinámica
Complejidad	$\mathcal{O}(V^2)$ o $\mathcal{O}(E \log V)$	$\mathcal{O}(V^3)$
Pesos Negativos	Falla estrepitosamente	Los soporta correctamente
Estructura	Colas de Prioridad	Matrices Bidimensionales
Uso ideal	Enrutamiento GPS (Google Maps)	Tablas de ruteo BGP, Logística

Conclusiones

1. **La herramienta correcta para el problema correcto:** No existe un *mejor algoritmo absoluto*. Dijkstra es rey en velocidad para consultas individuales en grafos masivos, pero Floyd es indispensable cuando necesitas el panorama completo o hay costos negativos.
2. **La elegancia de la Programación Dinámica:** Floyd-Warshall demuestra cómo 3 simples bucles `for` anidados pueden resolver un problema logístico a escala nacional, construyendo conocimiento desde subproblemas pequeños.

4. Asignaciones (Tarea)

Ejercicio 1: Maestría en Dijkstra

Enrutamiento de Drones

Dado el siguiente grafo de nodos (A, B, C, D, E) y enlaces bidireccionales: (A-B: 4), (A-C: 2), (B-C: 1), (B-D: 5), (C-D: 8), (C-E: 10), (D-E: 2).

1. Aplica el algoritmo de Dijkstra paso a paso comenzando desde el nodo **A**.
2. Muestra la tabla final de estados.
3. Reconstruye el camino más corto hacia **E**.

Ejercicio 2: Maestría en Floyd

Topología de Datacenter

Construye la matriz inicial y aplica el algoritmo de Floyd-Warshall para un pequeño cluster de 4 servidores (1, 2, 3, 4) con rutas dirigidas:

$(1 \rightarrow 2: 3)$, $(1 \rightarrow 4: 7)$, $(2 \rightarrow 1: 8)$, $(2 \rightarrow 3: 2)$, $(3 \rightarrow 1: 5)$, $(3 \rightarrow 4: 1)$, $(4 \rightarrow 1: 2)$.

1. Escribe la matriz de distancias final.
2. ¿Cuál es el camino más corto de 2 a 4 y de cuánto es su latencia?

Ejercicio 3: Hacia la Complejidad NP-Hard

Preparación para la próxima clase: *El problema del Viajero y la Mochila*

Plantea un modelo matemático inicial para la siguiente situación:

Eres un mensajero que debe salir de la base A, visitar los puntos B, C y D **exactamente una vez** y regresar a la base A (El Agente Viajero). Además, tu mochila solo soporta 10 kg de peso máximo, y cada paquete a entregar tiene un peso y una prioridad diferente (La Mochila).

Pregunta de reflexión: ¿Por qué Dijkstra o Floyd NO pueden resolver el problema de visitar TODOS los nodos en una sola ruta de manera óptima?